

Web API security

Web API security entails authenticating programs or users who are invoking a web API.

Along with the ease of API integrations come the difficulties of ensuring proper authentication (AuthN) and authorization (AuthZ). In a multitenant environment, security controls based on proper AuthN and AuthZ can help ensure that API access is limited to those who need (and are entitled to) it. Appropriate AuthN schemes enable producers (APIs or services) to properly identify consumers (clients or calling programs), and to evaluate their access level (AuthZ). In other words, may a consumer invoke a particular method (business logic) based on the credentials presented?

"Interface design flaws are widespread, from the world of crypto processors through sundry embedded systems right through to antivirus software and the operating system itself."^[1]

Method of authentication and authorization

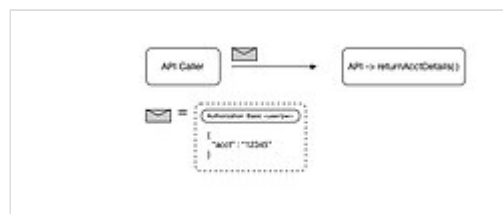
The most common methods for authentication and authorization include:

1. Static strings: These are like passwords that are provided by API's to consumers.
2. Dynamic tokens: These are time based tokens obtained by caller from an authentication service.
3. User-delegated tokens: These are tokens such as OAuth^[2] which are granted based on user authentication.
4. Policy & attribute-based access control: policies use attributes to define how APIs can be invoked using standards such as ALFA or XACML.

The above methods provide different level of security and ease of integration. Oftentimes, the easiest method of integration also offers weakest security model.

Static strings

In static strings method, the API caller or client embeds a string as a token in the request. This method is often referred as basic authentication.^[3] "From a security point of view, basic authentication is not very satisfactory. It means sending the user's password over the network in clear text for every single page accessed (unless a secure lower-level protocol, like SSL, is used to encrypt all transactions). Thus the user is very vulnerable to any packet sniffers on the net."^[4]



Basic Authentication Block Diagram



Dynamic tokens

When an API is protected by a dynamic token, there is a time-based nonce inserted into the token. The token has a time to live (TTL) after which the client must acquire a new token. The API method has a time check algorithm, and if the token is expired, the request is forbidden. "An example of such token is JSON Web Token. The "exp" (expiration time) claim identifies the expiration time on or after which the JWT MUST NOT be accepted for processing."^[5]

User-delegated token

This type of token is used in three-legged systems where an application needs to access an API on behalf of a user. Instead of revealing user id and password to the application, a user grants a token which encapsulates users permission for the application to invoke the API.

The OAuth 2.0 authorization framework enables a third-party application to obtain limited access to an HTTP service, either on behalf of a resource owner by orchestrating an approval interaction between the resource owner and the HTTP service, or by allowing the third-party application to obtain access on its own behalf.^[6]

Fine-Grained Authorization for APIs

Attribute-Based Access Control

In this approach, there is a Policy Enforcement Point either within the API itself, in the API framework (as an interceptor or message handler), or as an API gateway (e.g. WSO2, Kong, Tyk, or similar) that intercepts the call to the API and / or the response back from the API. It converts it into an authorization request (typically in XACML) which it sends to a Policy Decision Point (PDP). The Policy Decision Point is configured with policies that implement dynamic access control that can use any number of user, resource, action, and context attributes to define which access is allowed or denied. Policies can be about:

1. the resource (e.g. a bank account)
2. the user (e.g. a customer)
3. the context (e.g. time of day)
4. a relationship (e.g. the customer to whom the account belongs).

Policies are expressed in ALFA or XACML.

References

1. "API Attacks" (<https://www.cl.cam.ac.uk/~rja14/Papers/SEv2-c18.pdf>) (PDF).
2. "OAuth 2.0 — OAuth" (<http://oauth.net/2/>). *oauth.net*. Retrieved 2015-10-10.
3. "Hypertext Transfer Protocol -- HTTP/1.0: Basic Authentication Scheme" (<http://www.w3.org/Protocols/HTTP/1.0/spec.html#BasicAA>). *w3.org*. Retrieved 2026-07-08.
4. "A Guide to Web Authentication Alternatives: Part 2" (<http://unixpapa.com/auth/basic.html>). *unixpapa.com*. Retrieved 2015-10-10.

5. John, Bradley; Nat, Sakimura; Michael, Jones. "JSON Web Token (JWT)" (<https://tools.ietf.org/html/rfc7519>). *tools.ietf.org*. Retrieved 2015-10-10.
6. Hardt, Dick. "The OAuth 2.0 Authorization Framework" (<https://tools.ietf.org/html/rfc6749>). *tools.ietf.org*. Retrieved 2015-10-11.

External links

- [OWASP API Security Project \(https://owasp.org/www-project-api-security/\)](https://owasp.org/www-project-api-security/)
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Web_API_security&oldid=1363209152"